

Building Blocks of the Web

Complete Coding Guide & Worksheet: Building Your First React Student Dashboard

Subject: Web Development & Computer Science

Target Grade: Middle / High School

Tech Stack: React, JSX, Props, State

1. Introduction: How Web Applications Work

Have you ever wondered how school websites display dynamic content like individual student grades, live sports scores, or interactive notice boards? Modern web applications are built out of reusable, independent visual building blocks called **Components**.

The Lego Analogy

Think of web components like Lego bricks! Instead of manufacturing an entire toy house out of a single continuous piece of plastic, you assemble small, specialized Lego bricks (buttons, cards, banners) together to construct complex structures.

2. Step-by-Step React Setup & Environment

To start writing React code, developers use online code editors like **Vite + React**, **CodeSandbox**, or **StackBlitz**. Every React application begins with an entry point that renders the root component into the web browser DOM.

Step 1: Setting up main.jsx (Application Entry Point)

```
import React from 'react'; import ReactDOM from 'react-dom/client'; import App from './App'; //
Renders our main App component into the HTML div with id="root"
ReactDOM.createRoot(document.getElementById('root')).render( <React.StrictMode> <App /> </
React.StrictMode> );
```

3. Components and JSX Syntax

JSX (JavaScript XML) allows developers to write HTML-like markup directly inside JavaScript files. There are a few key JSX rules to remember:

- Use `className` instead of `class` for CSS styles.
- Wrap dynamic JavaScript expressions inside single curly braces `{ }`.
- Every component function **must return a single parent container tag** (e.g., `<div>` or React Fragment `<></>`).

4. Passing Dynamic Data with Props

Props (short for properties) allow parent components to pass data down to child components.

The Tiffin Box Analogy

Props are like school tiffin boxes passed into school every morning. The container box (`<StudentCard />`) stays the same, but the delicious food inside (`props.name` , `props.grade`) changes depending on who receives it!

Step 2: Creating the Reusable Child Component (StudentCard.jsx)

```
// Child Component receiving props
function StudentCard(props) {
  return (
    <div
      className="student-card">
      <h2>{props.name}</h2>
      <p><strong>Grade:</strong> {props.grade}</p>
      <p><strong>House:</strong> {props.house}</p>
      <p><strong>Score:</strong> {props.score} pts</p>
    </div>
  );
}
export default StudentCard;
```

5. Complete Interactive App Code (State & Interactivity)

To make our dashboard interactive (e.g. adding house points with a button click), React uses a special Hook called `useState`.

Step 3: Building the Complete Dashboard Application (App.jsx)

```
import React, { useState } from 'react';
import StudentCard from './StudentCard';

function App() {
  // Managing interactive state for score
  const [score, setScore] = useState(85);
  // Function to increment student points
  const handleAddPoints = () => {
    setScore(score + 5);
  };

  return (
    <div
      className="dashboard">
      <h1>School Student Dashboard</h1>
      {/* Passing props dynamically from parent to children */}
      <StudentCard
        name="Aarav Sharma"
        grade="8-A"
        house="Agni"
        score={score}
      />
      <StudentCard
        name="Ananya Patel"
        grade="8-B"
        house="Trishul"
        score={92}
      />
      {/* Interactive Button to trigger state change */}
      <button
        onClick={handleAddPoints}
        className="btn-add">
        + Add 5 Bonus Points to Aarav
      </button>
    </div>
  );
}

export default App;
```

6. Real-World React Applications in Schools

School Application	React Components Used	Dynamic Props & State Passed
Digital Notice Board	<code><AnnouncementCard /></code>	<code>title</code> , <code>date</code> , <code>category</code>
Online Report Cards	<code><SubjectGrade /></code>	<code>subject</code> , <code>marks</code> , <code>grade</code>
Cricket Scoreboards	<code><LiveScore /></code>	<code>runs</code> , <code>wickets</code> , <code>overs</code> (State)
House Leaderboard	<code><HouseTally /></code>	<code>houseName</code> , <code>points</code>

STUDENT WORKSHEET & CODING EXERCISES

Name: _____ Date: _____

SECTION A: Conceptual Multiple Choice

1. What is a React Component best compared to?

- (a) A video game controller
- (b) A Lego building brick
- (c) A complex math formula
- (d) A computer hardware cable

2. What attribute is used in JSX to assign CSS class names instead of HTML's `class` ?

- (a) `styleClass`
- (b) `className`
- (c) `cssName`
- (d) `class`

3. How do you pass dynamic JavaScript expressions or variables into JSX markup?

- (a) Inside quotation marks `"var"`
- (b) Inside curly braces `{var}`
- (c) Inside angle brackets `<var>`
- (d) Inside parentheses `(var)`

4. How would you pass the student name "Aarav" as a prop into `<StudentCard />` ?

- (a) `<StudentCard name="Aarav" />`
- (b) `<StudentCard props.name="Aarav">`
- (c) `StudentCard("Aarav")`
- (d) `<StudentCard -> Aarav>`

SECTION B: Code Completion Exercises (Fill in the Blanks)

5. Complete the missing React code snippets below:

a) Complete the function definition to accept props and display the student's name:

```
function NoticeBoard( _____ ) {  
  return <h1>Notice for: { _____ .title}</h1>;  
}
```

b) Complete the JSX prop assignment to send a score of `100` to a component:

```
<CricketScoreboard runs={ _____ } team=" _____ " />
```

SECTION C: Code Analysis & Debugging Challenge

6. Spot the 3 Bugs! The code block below contains 3 syntax errors. Identify and fix them:

```
// BUGGY CODE: function ReportCard(props) { return ( <div class="card"> <h2>Student:
props.studentName</h2> <p>Marks: {props.marks}</p> </div> <button>Print Report</
button> ); }
```

Write your bug fixes below:

Bug 1:

Bug 2:

Bug 3:

SECTION D: Step-by-Step Hands-On Coding Challenge

Home / Lab Practical: Build a House Points Score Tracker Component

Goal: Create a brand-new component named `<HouseLeaderboard />` in CodeSandbox or StackBlitz.

Step-by-Step Instructions:

1. Create a new file named `HouseLeaderboard.jsx`.
2. Define a functional component that accepts `props` (`houseName`, `captain`, `points`).
3. Return a container `<div className="house-box">` styled with heading and paragraphs.
4. Import `HouseLeaderboard` inside `App.jsx` and render four houses: *Agni*, *Trishul*, *Prithvi*, and *Jal* with unique point values.

7. Write your code solution for `HouseLeaderboard.jsx` below:

```
function HouseLeaderboard(props) {
```

```
  return (
```

```
  );
```

```
}
```

```
export default HouseLeaderboard;
```